Synopsis

Project Title

Problem Id: 16

Student Name: Vineet Bhatt

University Roll number: 2028908

Section: K

Student Name: Yukti Verma

University Roll number: 2028914

Section: K

Student Name: Kushagra Rawat

University Roll number: 2028752

Section: K

1. Problem Description:

This problem involves designing an *end-to-end machine learning system* that predicts the *sentiment (opinion)* of a product review written by a user. The system takes *raw review text* as input and classifies it into three categories: *Positive, Negative, or Neutral*.

First, the text data is *preprocessed* to remove unnecessary elements such as HTML tags, punctuation, and extra spaces. Then *tokenisation, **stop-word removal, and **stemming/lemmatisation* are applied to clean and simplify the text. After preprocessing, the text is converted into *numerical features* so that machine learning models can understand it.

Next, *feature extraction and feature selection* are used to keep only the most important words or patterns. The system then trains multiple classifiers such as *Logistic Regression, K-Nearest Neighbours (KNN), and Support Vector Machine (SVM). The models are evaluated using metrics like **accuracy, precision, recall, F1-score, confusion matrix, and ROC curve* to determine which model performs best.

***Real-life role:-**

Sentiment analysis is widely used by *e-commerce companies* to understand customer opinions about products. It helps businesses improve product quality, monitor customer satisfaction, detect negative feedback quickly, and make better marketing decisions. Platforms like online shopping sites, movie review sites, and social media use such systems to analyse large volumes of user feedback automatically.

Steps of Implementation:

1. Data Collection

In this phase, a dataset of product reviews is collected. The dataset contains review text and corresponding sentiment labels (positive, negative, neutral). Data may be obtained from e-commerce platforms, online review datasets, or public ML repositories. Good quality and sufficient quantity of data are important for building an accurate model.

2. Data Preprocessing

Raw review text often contains noise such as HTML tags, punctuation, and unnecessary words. Therefore, the data is cleaned using preprocessing techniques like:

Removing HTML tags and punctuation

Tokenization (splitting text into words)

Stop-word removal (removing words like is, the, and)

Stemming or Lemmatization to convert words to their root form

Converting text into numerical vectors using methods like Bag of Words or TF-IDF

This step prepares the data so that machine learning algorithms can process it effectively.

3. Model Selection

In this phase, suitable machine learning algorithms are selected for sentiment classification. Commonly used models include:

Logistic Regression

K-Nearest Neighbors (KNN)

Support Vector Machine (SVM)

Multiple models are chosen to compare their performance.

4. Training and Validation

The dataset is divided into training and validation sets.

During training, models learn patterns between review text and sentiment labels.

During validation, the trained models are tested on unseen data to check their generalization ability and avoid overfitting.

5. Evaluation

The performance of each model is measured using evaluation metrics such as:

Accuracy

Precision

Recall

F1-Score

Confusion Matrix

ROC Curve

These metrics help determine which model performs best.

6. Final Model:

After evaluation, the model with the best performance is selected as the final model. This model is then used to automatically predict the sentiment of new product reviews.

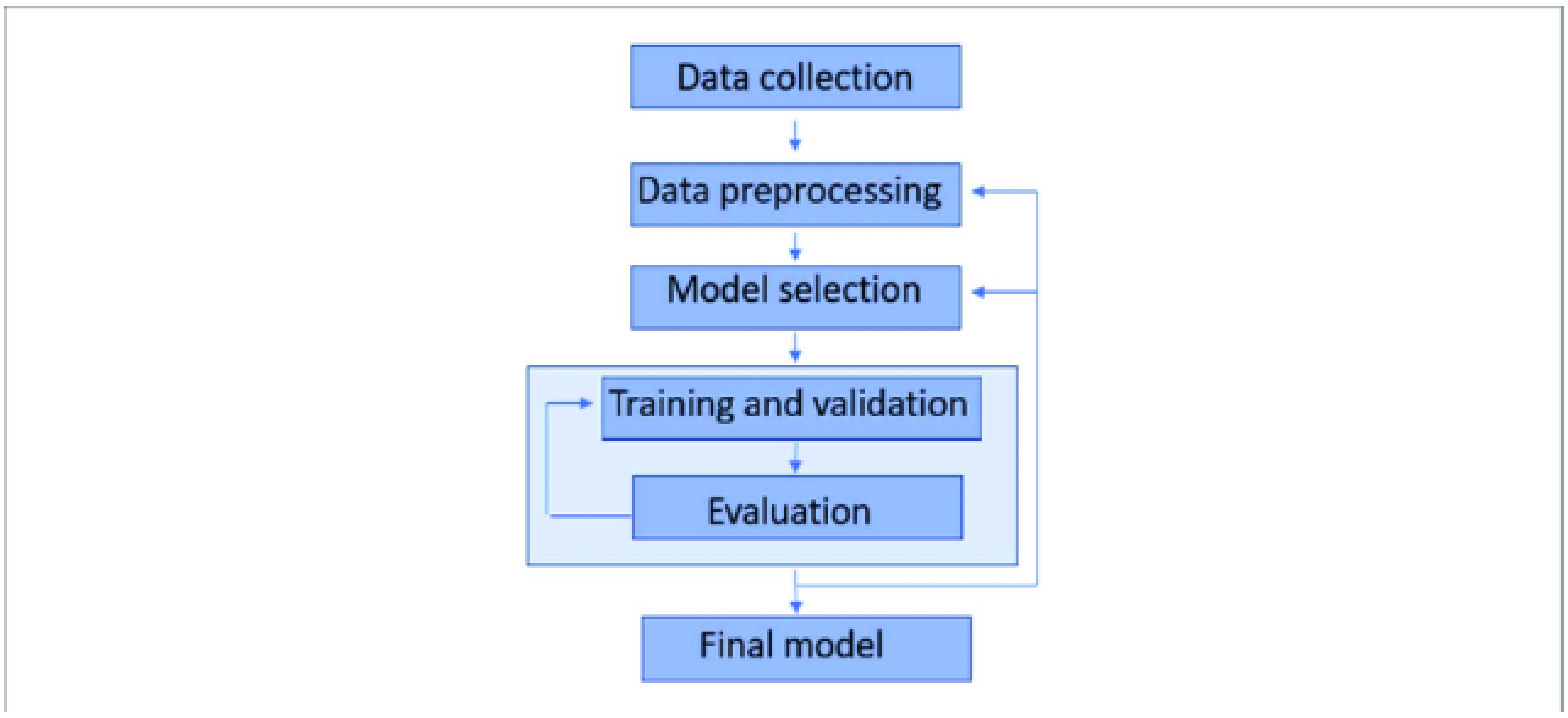

Fig. 1 Sample architecture

2. Expected Modules/ Libraries to be used :

1. NumPy

Used for numerical operations and array handling. It helps in performing mathematical computations on numerical feature vectors generated from text data.

2. Pandas

Used for data manipulation and analysis. It helps load datasets (CSV files), organize review data in dataframes, and perform operations like filtering, cleaning, and handling missing values.

3. NLTK

A Natural Language Processing library used for text preprocessing tasks such as tokenization, stop-word removal, stemming, and lemmatization of product review text.

4. re

An inbuilt Python module used for pattern matching and text cleaning, such as removing HTML tags, punctuation, and unwanted characters from reviews.

5. Scikit-learn

A widely used machine learning library that provides tools for feature extraction, model training, and evaluation. It includes algorithms like Logistic Regression, KNN, and SVM used for sentiment classification.

6. Matplotlib

A plotting library used for data visualization, such as displaying graphs like ROC curves and performance plots of the machine learning model.

7. Seaborn

A statistical visualization library built on Matplotlib that helps create better visualizations such as confusion matrix heatmaps for evaluating model performance.

3. Expected Evaluation metrics and Plots to be used:

Evaluation Metrics

1. Accuracy

Accuracy measures the overall correctness of the model by calculating the ratio of correctly predicted reviews to the total number of reviews. It provides a general idea of how well the classifier performs on the dataset.

2. Precision

Precision measures the correctness of positive predictions. It shows how many of the reviews predicted as a particular sentiment (e.g., positive) are actually correct. This is useful when false positive predictions need to be minimized.

3. Recall

Recall measures the ability of the model to correctly identify all relevant instances of a sentiment class. It indicates how well the model detects all actual positive, negative, or neutral reviews.

4. F1-Score

The F1-score is the harmonic mean of precision and recall. It provides a balanced measure when both false positives and false negatives are important, making it suitable for sentiment classification tasks.

5. Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions for each class (Positive, Negative, Neutral). It helps analyze where the model is making mistakes and which sentiments are being misclassified.

4. Required topics from the subject:

Strings and Regular Expressions: Since the input is raw review text, string manipulation is the core of the project. You need to master Python’s str methods and the re module to perform tasks like case folding, removing HTML tags, and filtering out punctuation.

Data Structures (Lists and Dictionaries): These are essential for managing tokens (words) after text splitting. You will use lists to store sequences of words and dictionaries to map these words to their frequency or numerical weights during the vectorization process.

File Handling: This is required to import the dataset and export the final results. You must know how to use Pandas or Python’s built-in open() function to read CSV/text files and save the trained model weights for future use.

Object-Oriented Programming (OOP) / Functions: To keep the code modular, you should wrap the preprocessing, training, and evaluation steps into functions or classes. This allows you to pass different models (like SVM or KNN) through the same evaluation pipeline easily.

5. Platform Required:

Jupyter Notebook: This is the highly recommended platform for this project. Its cell-based structure allows you to visualize dataframes, plots (like ROC curves), and clean text step-by-step, which is vital for machine learning experimentation.

6. Books and Link Sources:

"Python Machine Learning" by Sebastian Raschka: A comprehensive guide that covers the implementation of Scikit-learn models and sentiment analysis workflows.

"Natural Language Processing with Python" by Steven Bird: The definitive book for learning the NLTK library and text preprocessing techniques.

Scikit-learn Official Documentation: scikit-learn.org – The best resource for understanding the parameters of Logistic Regression, SVM, and evaluation metrics.

Kaggle Datasets and Kernels: kaggle.com – An excellent source for finding "Product Review" datasets and seeing how other data scientists handle text cleaning.